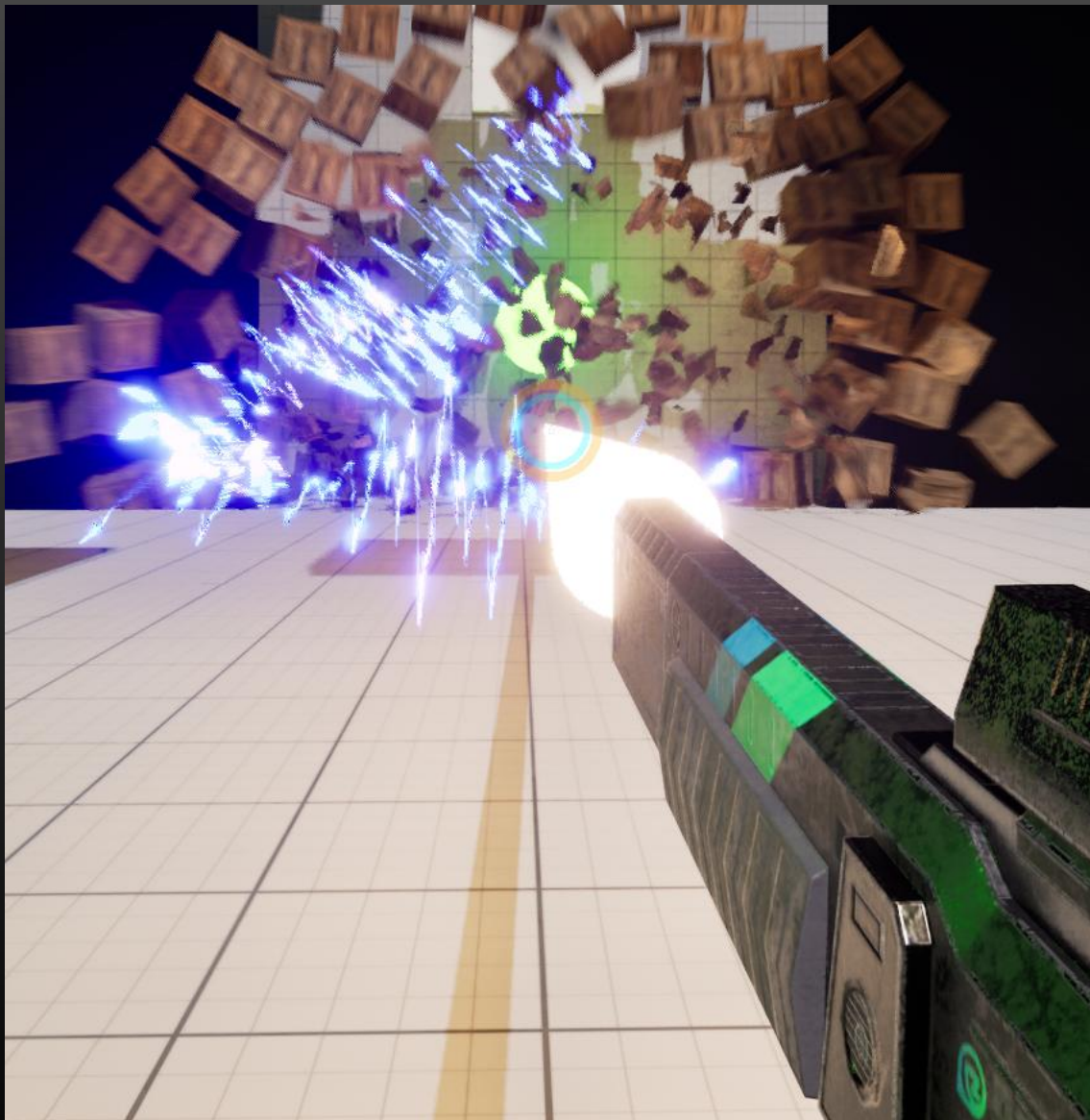




Railgun FPS v1.0

拡張性のある設計による**柔軟性**と、
自作ツールによる**効率化**を体感

東京情報デザイン専門職大学
情報デザイン学部 情報デザイン学科 3年
岸田 匠馬

プレイ動画URL : <https://youtu.be/nzGcXKg-ckQ>

ゲーム概要



Railgun FPS v1.0 ジャンル：FPS

レールガンの破壊力を活かして
各ステージをクリアしていくゲーム

- ・レールガンは2段階チャージできる。
チャージするほど飛距離と威力が上がる
- ・v1.0では「的当てステージ」のみ実装
- ・すべての木箱を時間内に破壊したらクリア

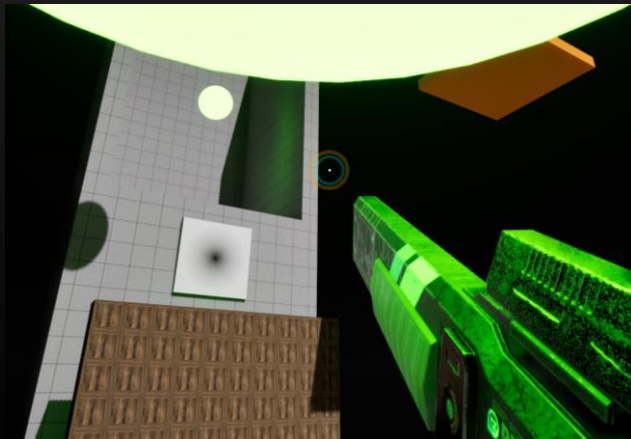
開発環境・スペック

開発人数	1人
開発期間	3ヶ月
使用ツール	Unreal Engine 5/BluePrint/C++, Visual Studio
プラットフォーム	Windows
担当範囲	すべて

ゲーム概要（ギミック）

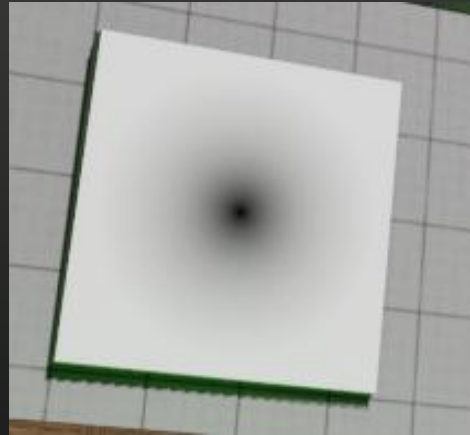
引き寄せギミック（仮）

当てると的のすぐ下まで引き寄せられる。



破壊可能壁ギミック（仮）

耐久が高く、橙フルチャージでないと一撃で壊せない。



青フルチャージ

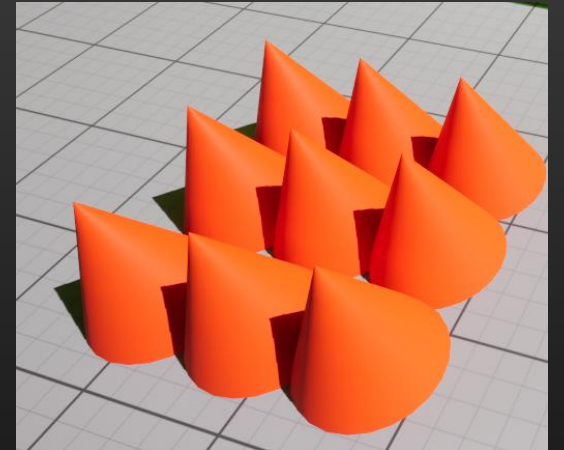


橙フルチャージ



とげギミック（v1.0では未配置）

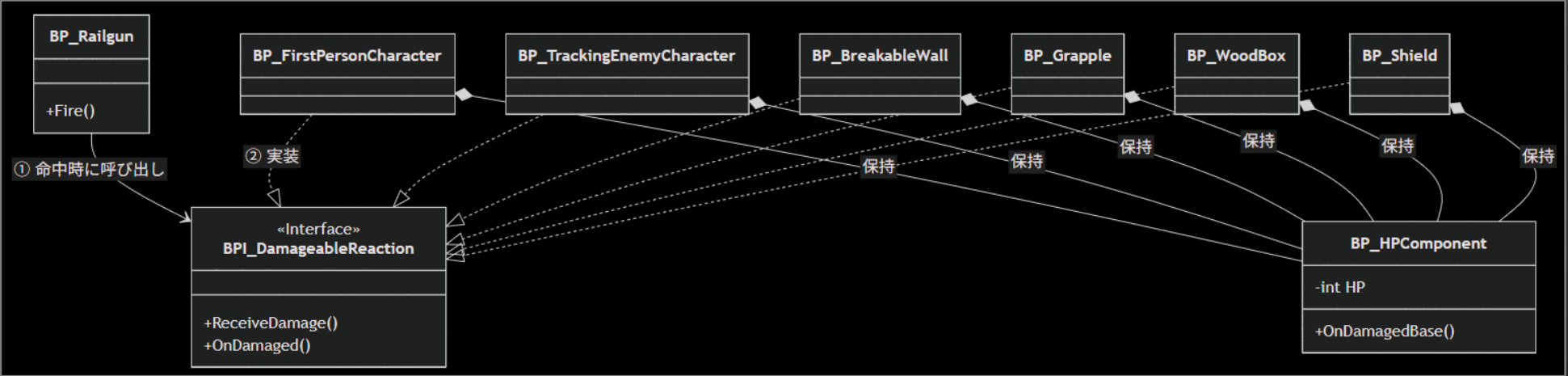
この上に立っているとダメージを少しずつ食らい続ける



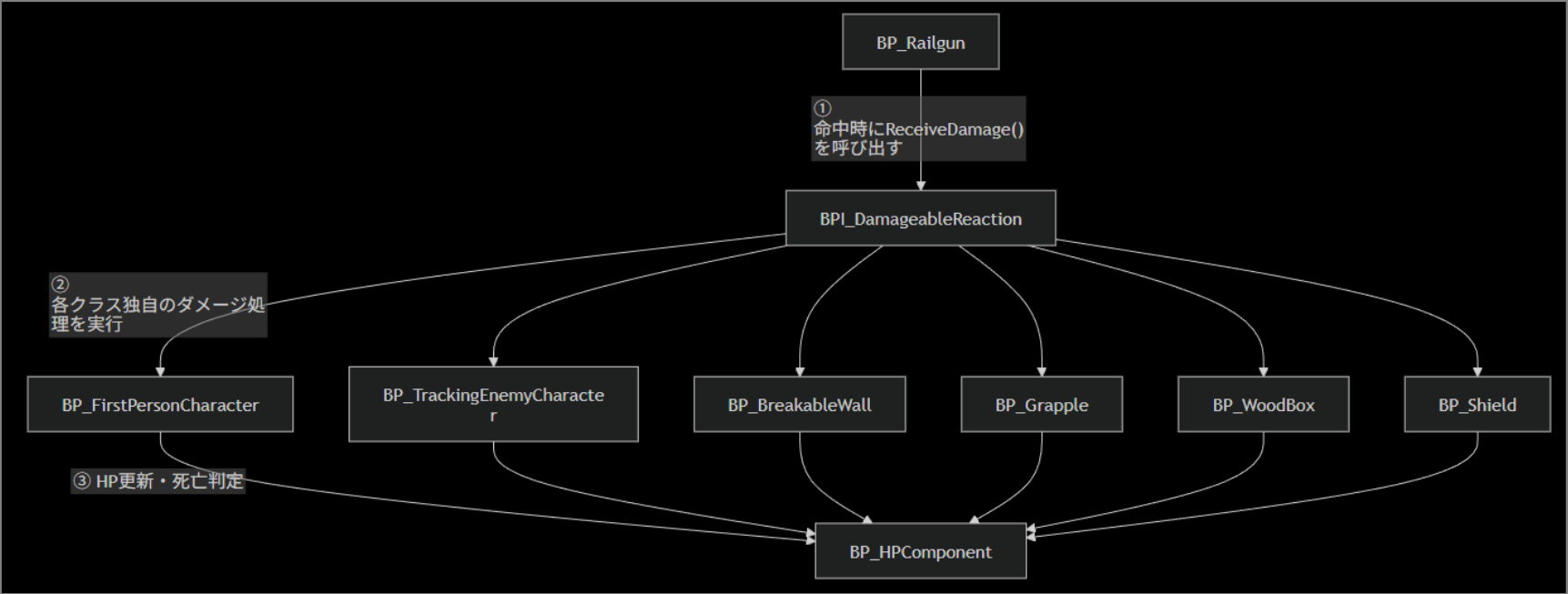
工夫点① オブジェクト指向

1. ダメージ処理設計（構成）

クラス図



処理フロー

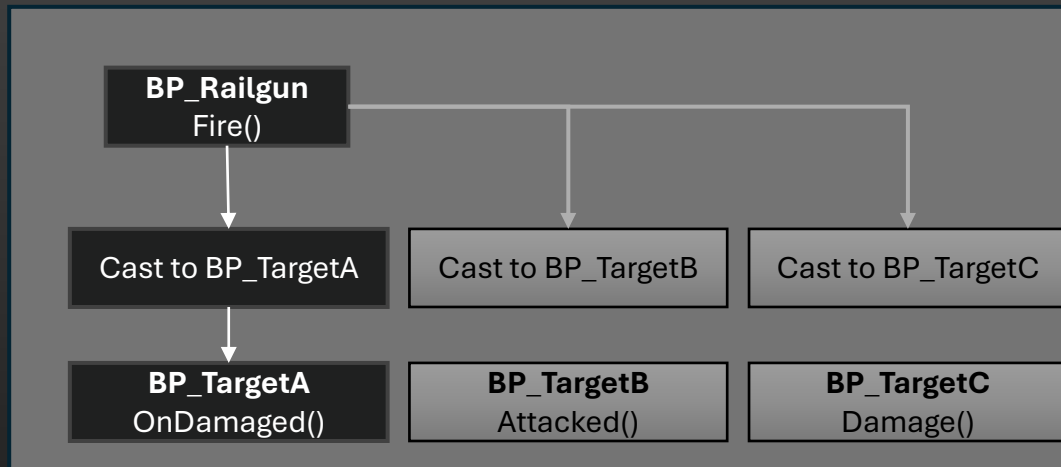


工夫点① オブジェクト指向

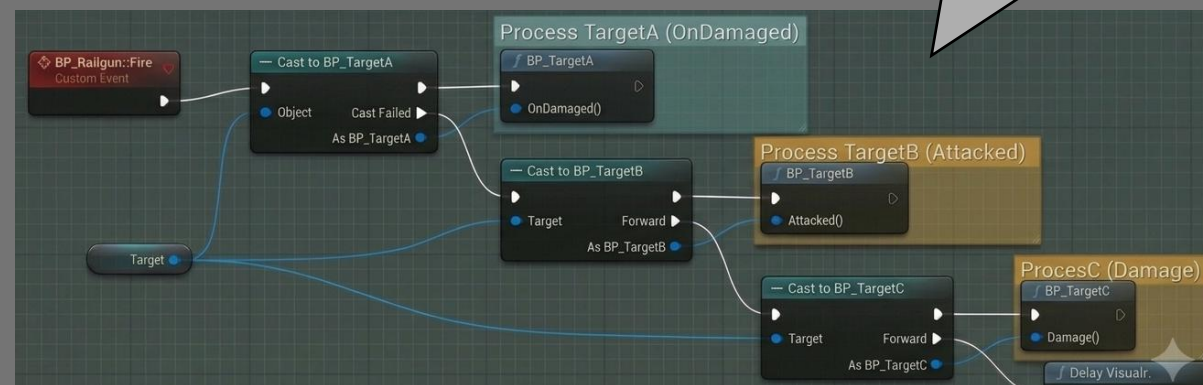
1. ダメージ処理設計（インターフェース）

- ・分岐が増え続ける
- ・複数の関数呼び出しを記述する必要がある

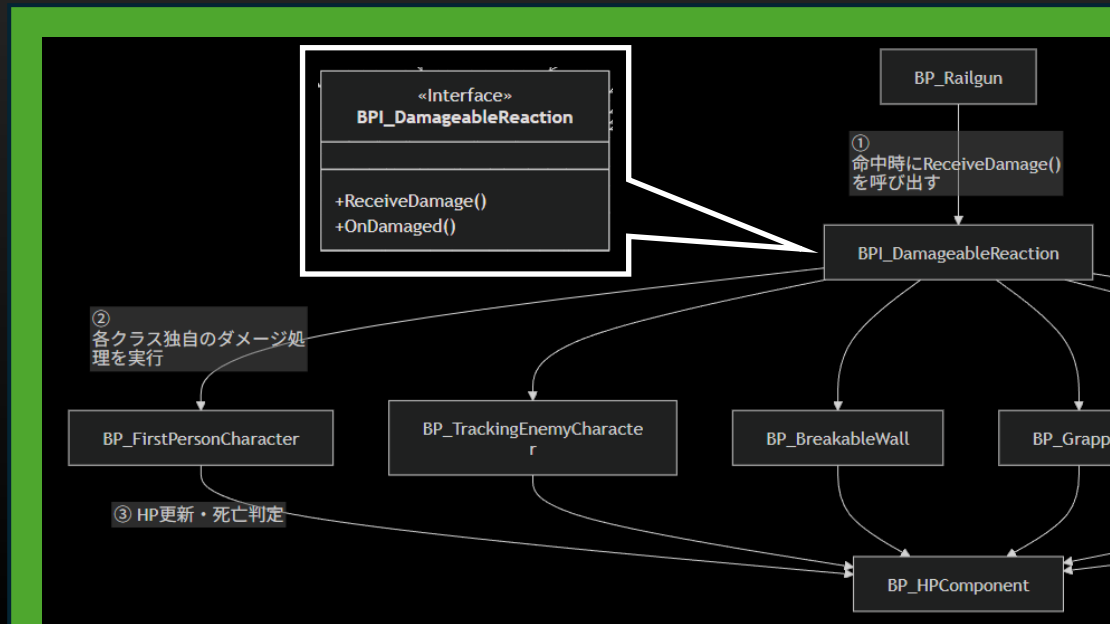
改善前



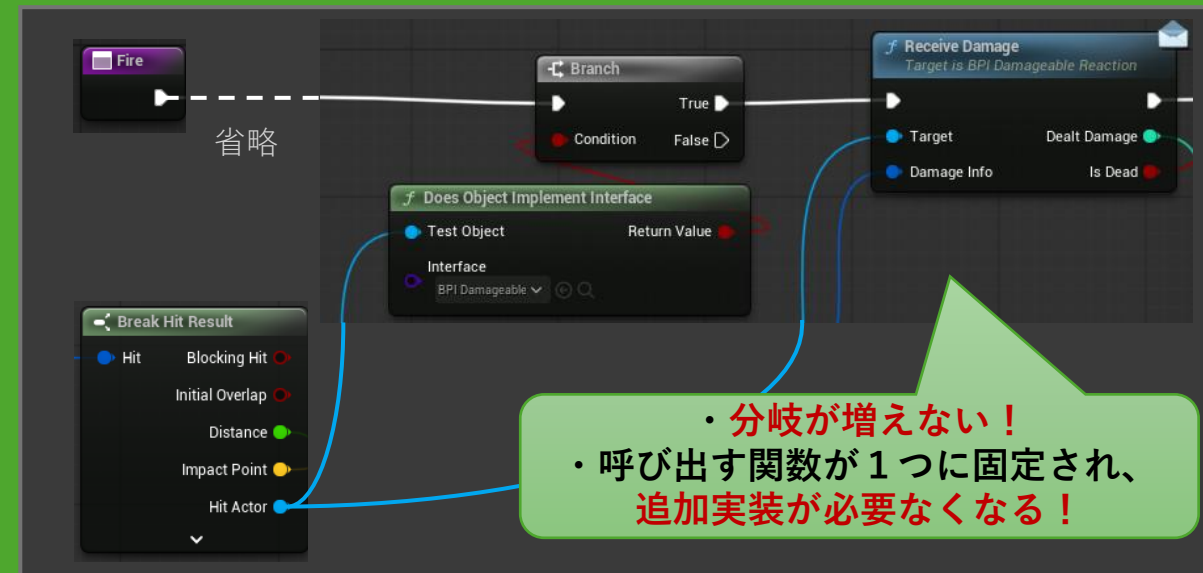
旧ダメージ処理に近いもの（Geminiで生成）



改善後



BP_Railgun.Fire()のダメージ処理部分



- ・分岐が増えない！
- ・呼び出す関数が1つに固定され、追加実装が必要なくなる！

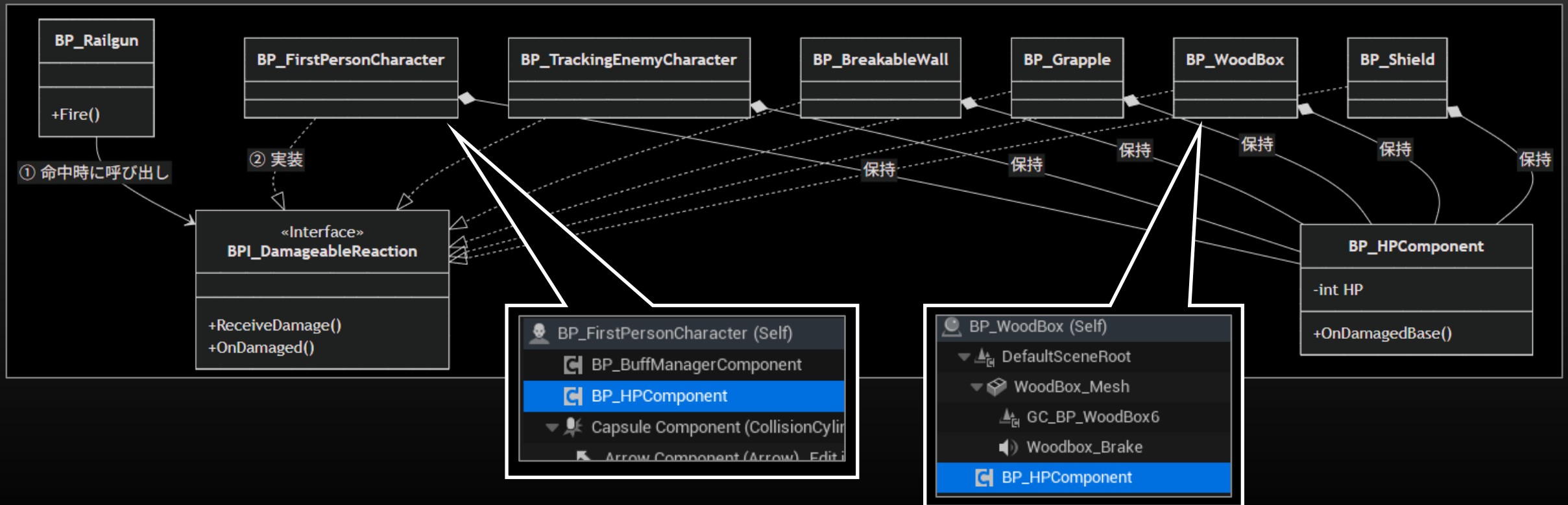
工夫点① オブジェクト指向

1. ダメージ処理設計（コンポーネント指向）

HP Component

HPの処理はどのActorにも共通の機能として存在していたため、1つのコンポーネントに分け、それぞれのActorに保持させた。

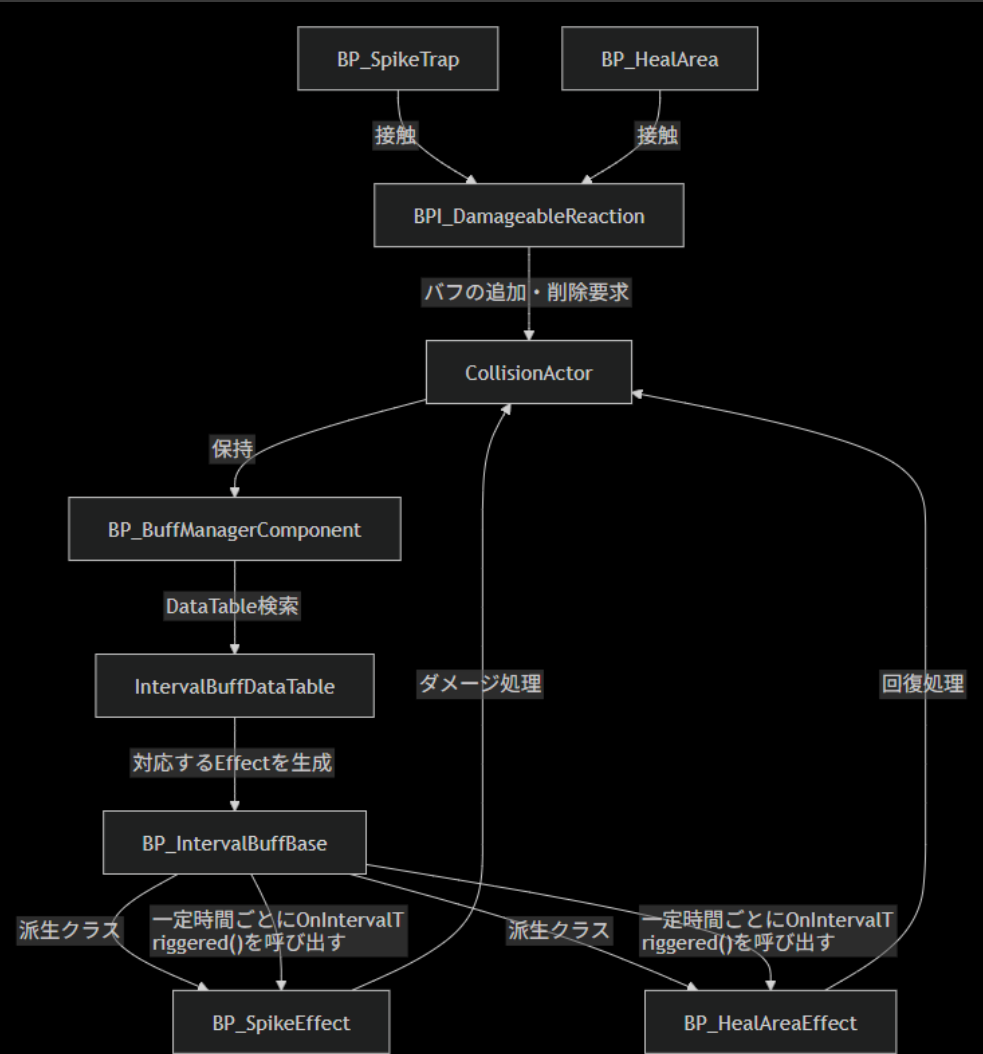
クラス図（ダメージ処理）



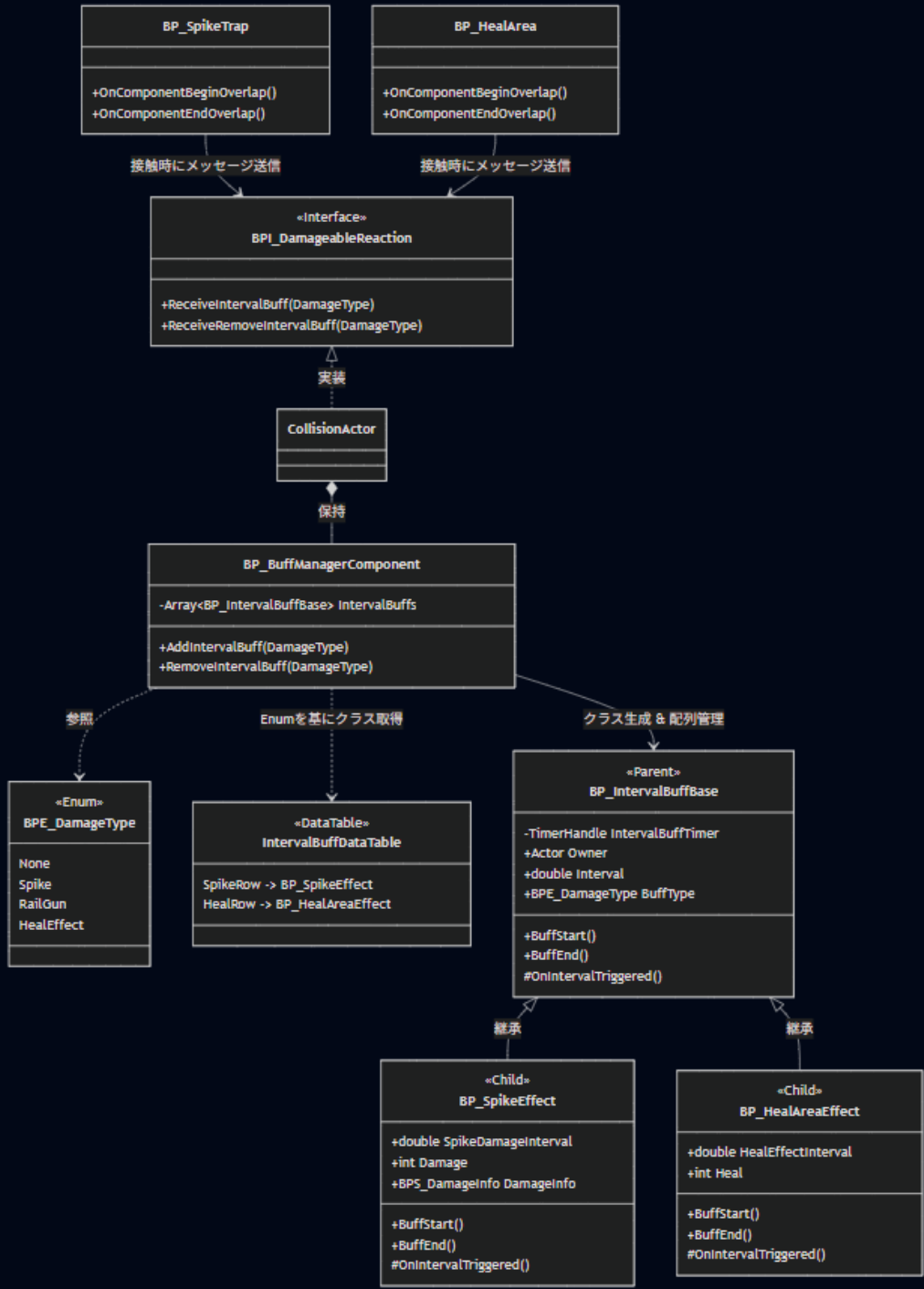
工夫点① オブジェクト指向

2. Buff処理の設計（構成）

処理フロー



クラス図



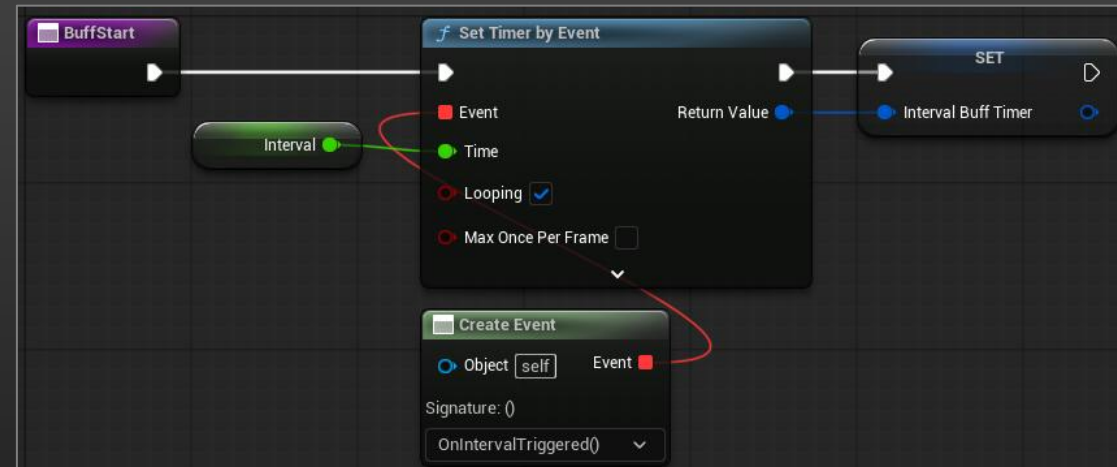
工夫点① オブジェクト指向

2. Buff処理の設計 (継承)

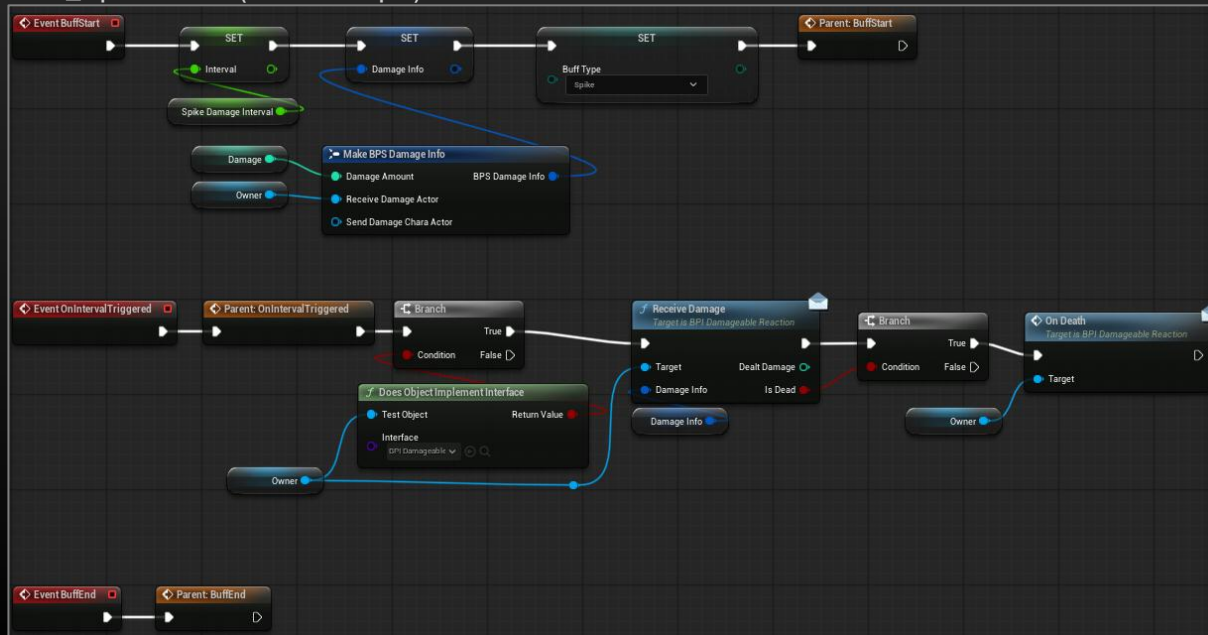
ダメージ処理と同様、
バフの種類が増えても複雑化しない設計で実装
ただし**共通処理**があるため**継承**を利用

派生クラスは「Start」「IntervalTriggered」「End」を記述
するだけになり、**コード(BP)**が非常に見やすくなった。

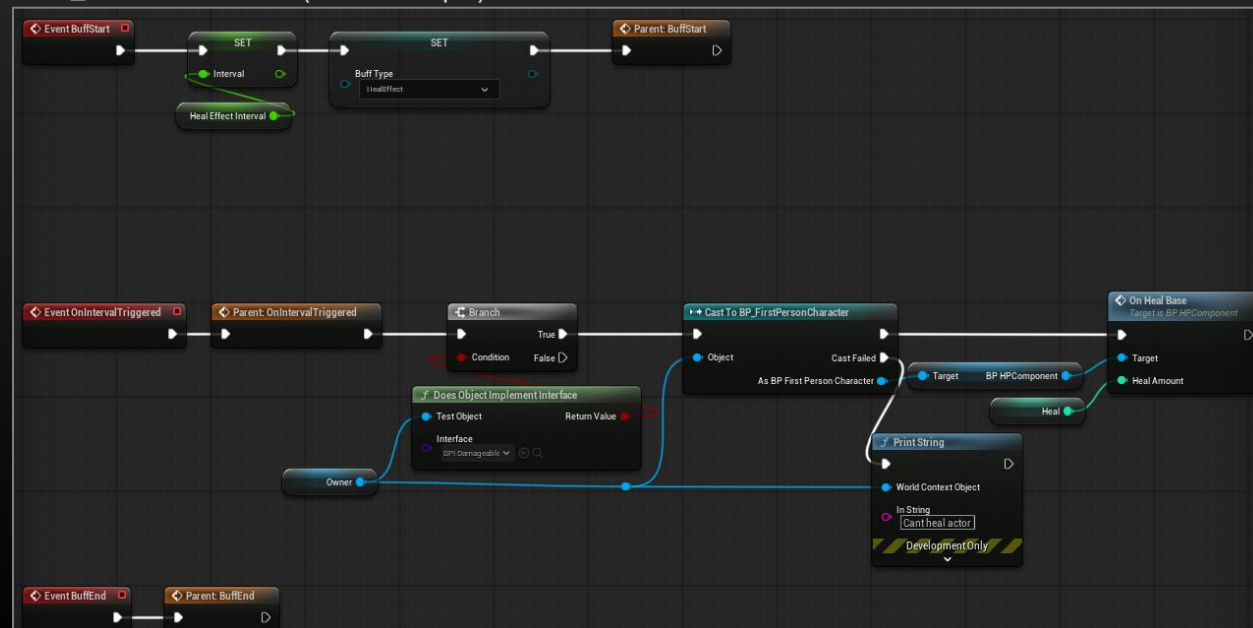
BP_IntervalBuffBase.BuffStart()



BP_SpikeEffect(Event Graph)



BP_HealAreaEffect (Event Graph)



工夫点① オブジェクト指向

2. Buff処理の設計 (コンポーネント指向)

Buffの全体管理を
BP_BuffManagerComponentにまとめること
で、このコンポーネントを付けるだけで
Buffが使用可能になる設計を実装した

BP_BuffManagerComponentに定義されている関数・変数

FUNCTIONS (2 OVERRIDABLE) Override ▾ +

f

GetIntervalBuffClass

f

AddIntervalBuff

f

RemoveIntervalBuff

f

GetCalcuDamage

f

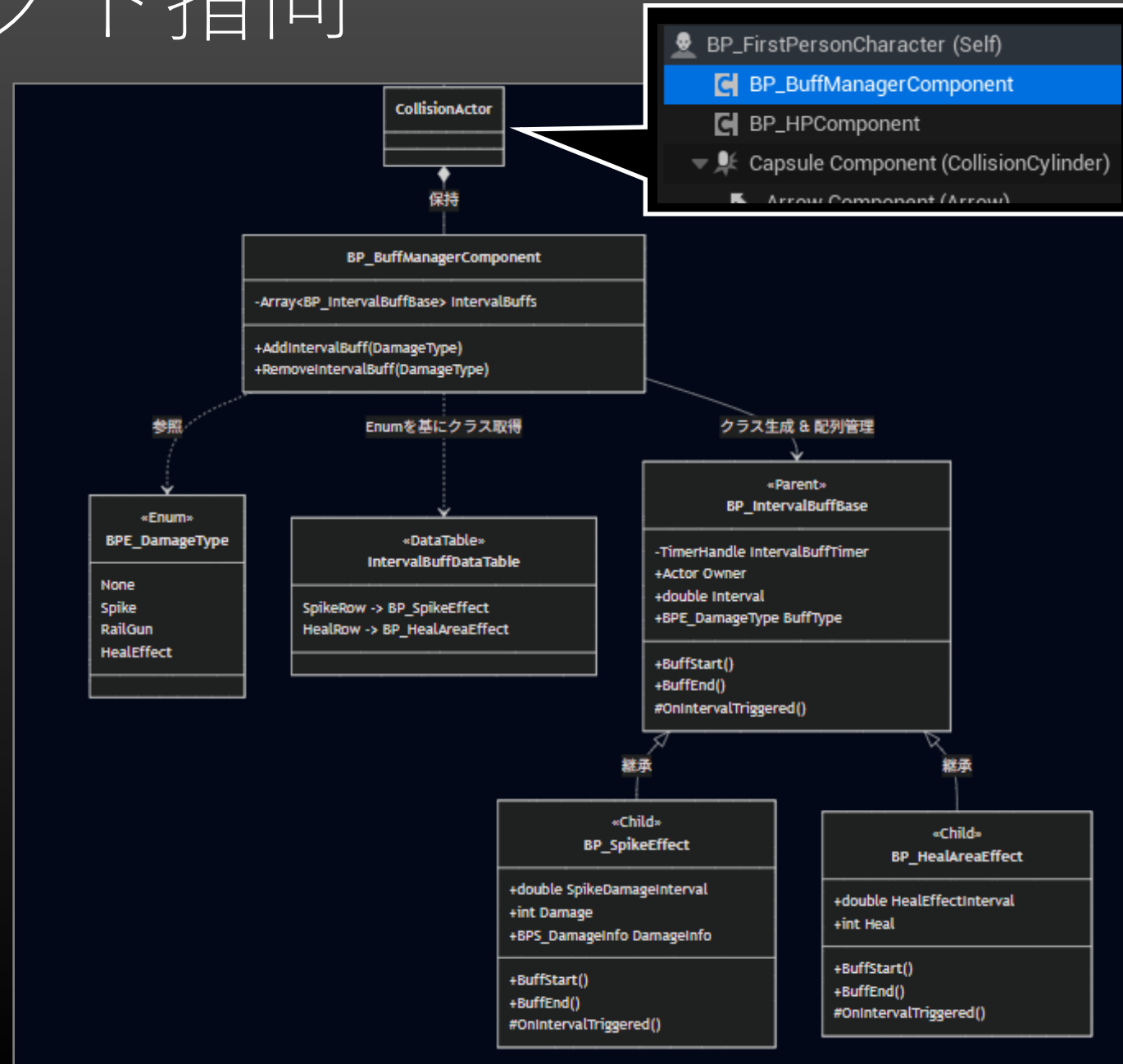
FindDamageTypeBuff

MACROS +

VARIABLES

IntervalBufs

 BP Interval Buff Base



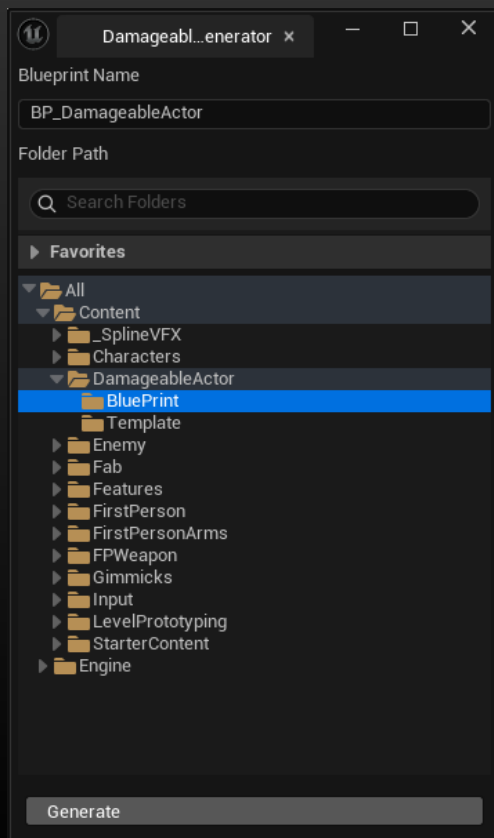
工夫点② Damageable Actor Generator ツール

ツールのGenerateボタンを押すだけでレールガンでダメージを受けるActorを生成できるツール

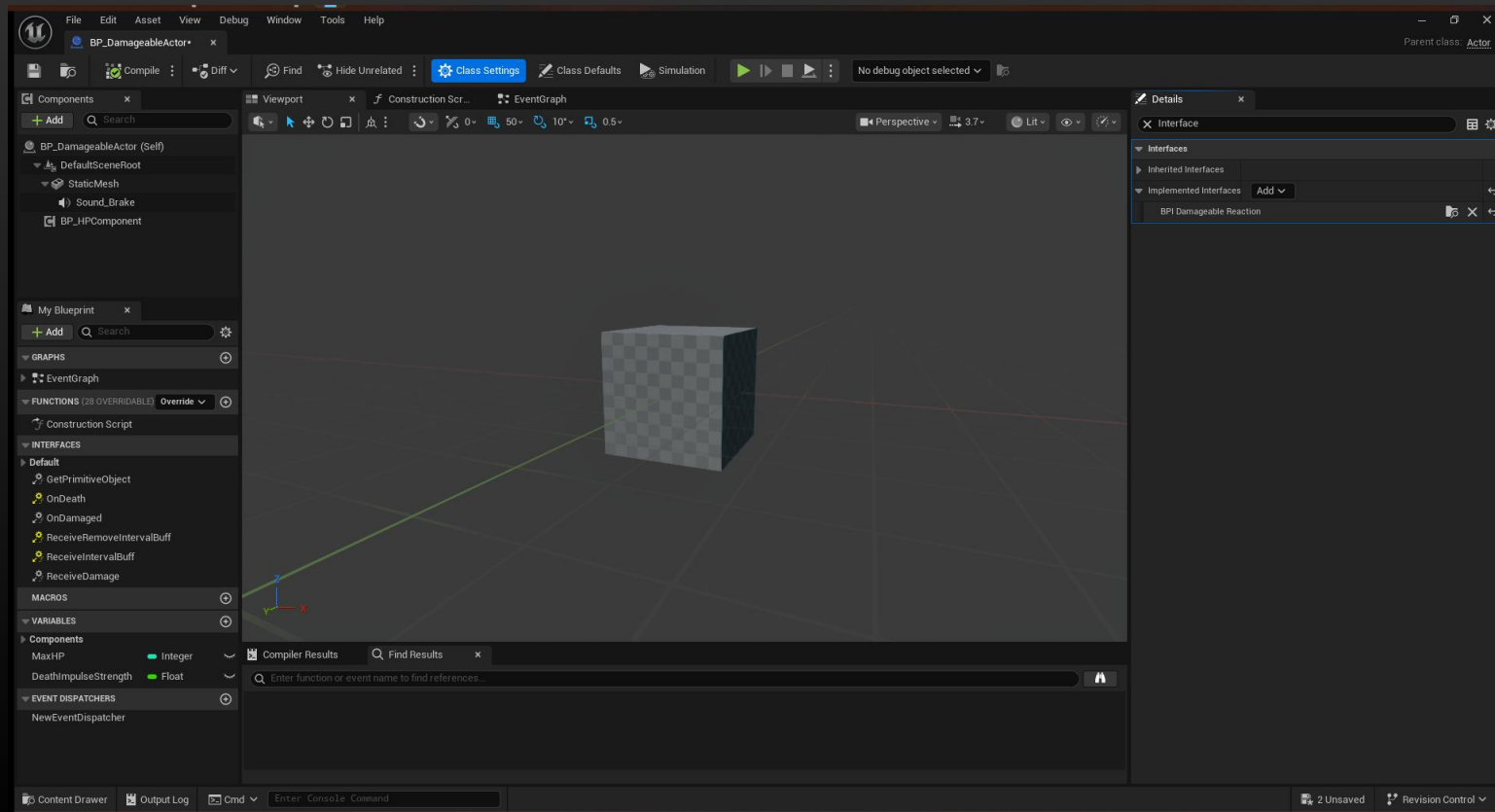
もともとオブジェクト指向（BPI_DamageableReaction）
やコンポーネント指向（BP_HPComponentなど）を用い
た設計にしていたため、追加処理がシンプルになった。



拡張性のある設計の柔軟性を体感！！



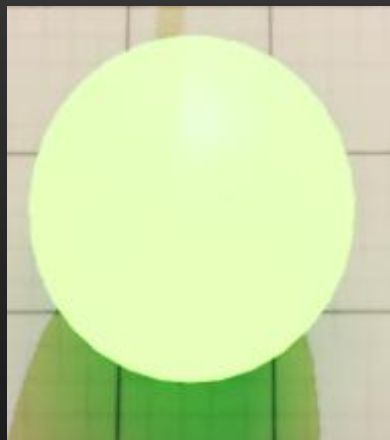
生成



工夫点② Damageable Actor Generator ツール

このツールで制作したもの

BP_Grapple



BP_BrakableWall



マテリアル制作時間を除けば
0から作るのに比べ、約10~20分ほど短縮できた。



ツールによる効率化を体感！
同時に達成感も！！

DamageableActorBPGenerator.cpp

```
238 UBlueprint* FDamageableActorBPGeneratorModule::CreateBlueprintFromTemplate(  
239     const FString& NewBPName,  
240     FString NewFolderPath)  
241 {  
242     // フォルダパスを正規化 (末尾の / を除去)  
243     NewFolderPath.RemoveFromEnd(TEXT("/"));  
244  
245     // Blueprint名をUEで使用可能な名前に変換  
246     const FString BaseAssetName = ObjectTools::SanitizeObjectName(NewBPName);  
247  
248     // AssetTools取得  
249     FAssetToolsModule& AssetToolsModule =  
250         FModuleManager::LoadModuleChecked<FAssetToolsModule>("AssetTools");  
251  
252     // 重複しない名前を自動生成  
253     FString PackageName;  
254     FString AssetName;  
255  
256     AssetToolsModule.Get().CreateUniqueAssetName(  
257         NewFolderPath + TEXT("/") + BaseAssetName,  
258         TEXT(""),  
259         PackageName,  
260         AssetName  
261     );  
262  
263     // テンプレートBPを読み込む  
264     UBlueprint* TemplateBP = LoadObject<UBlueprint>(  
265         nullptr,  
266         TemplatePath  
267     );  
268  
269     if (!TemplateBP)  
270     {  
271         UE_LOG(LogTemp, Error, TEXT("Failed to load BP_Template."));  
272         return nullptr;  
273     }  
274  
275     // Blueprint複製  
276     UObject* NewAsset = AssetToolsModule.Get().DuplicateAsset(  
277         AssetName,  
278         NewFolderPath,  
279         TemplateBP  
280     );  
281  
282     UBlueprint* NewBP = Cast<UBlueprint>(NewAsset);  
283  
284     if (!NewBP)  
285     {  
286         UE_LOG(LogTemp, Error, TEXT("Failed to duplicate template."));  
287         return nullptr;  
288     }  
289  
290     // 変更を反映  
291     NewBP->Modify();  
292     NewBP->MarkPackageDirty();  
293     FKismetEditorUtilities::CompileBlueprint(NewBP);  
294  
295     UE_LOG(LogTemp, Display, TEXT("Created Blueprint : %s"), *NewBP->GetName());  
296  
297     return NewBP;  
298 }
```

工夫点③ レールガンの射撃演出

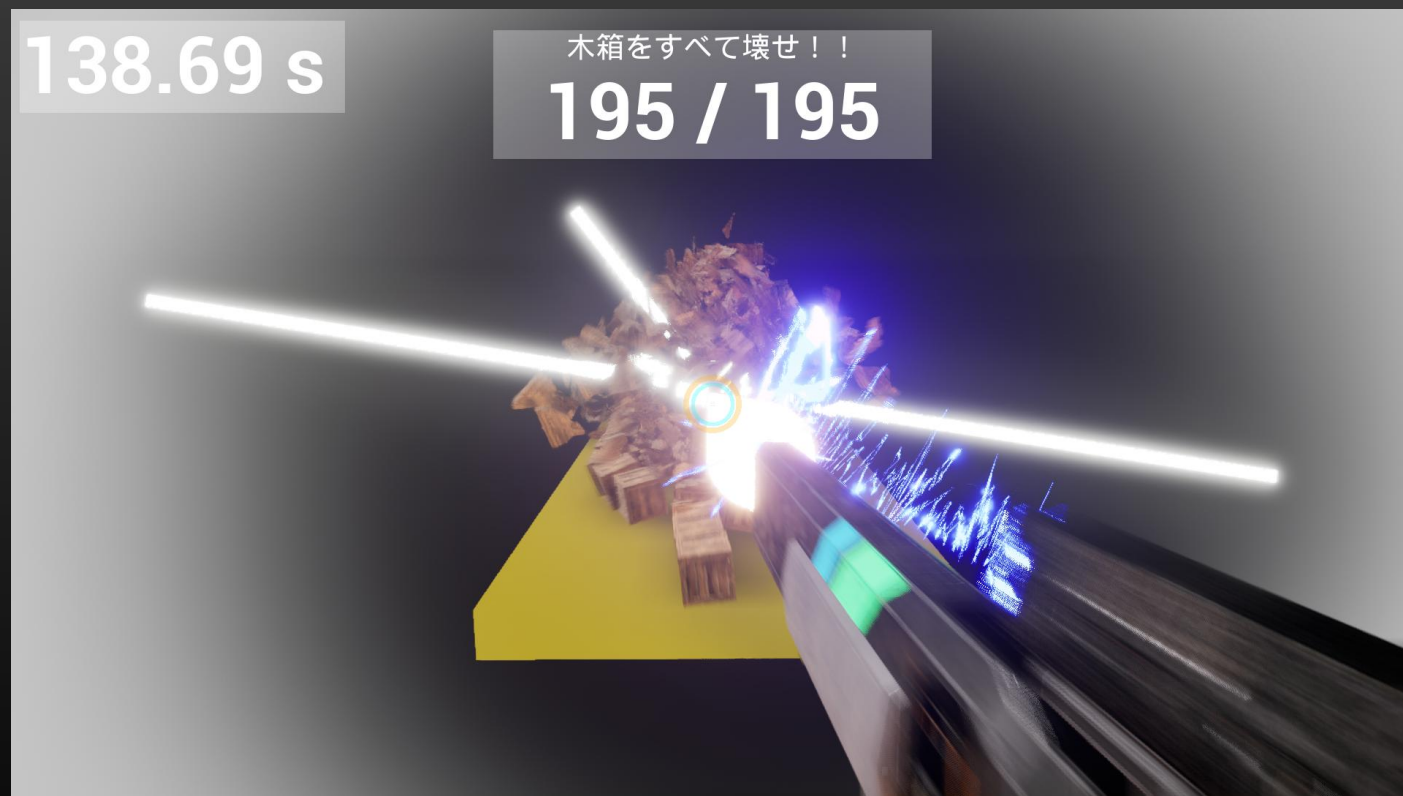
レールガンらしい重量感と破壊力を演出

高火力武器としての迫力を表現するため、
画面・武器・エフェクトを組み合わせた演出
を実装

- ・画面シェイク
- ・銃の反動
- ・衝撃波
- ・弾道エフェクト
- ・プラズマ表現
- ・着弾エフェクト
- ・木箱の破壊演出

特にこだわった

「プラズマ表現」「着弾エフェクト」「木箱の破壊演出」
の3つをこの後詳しく説明する。



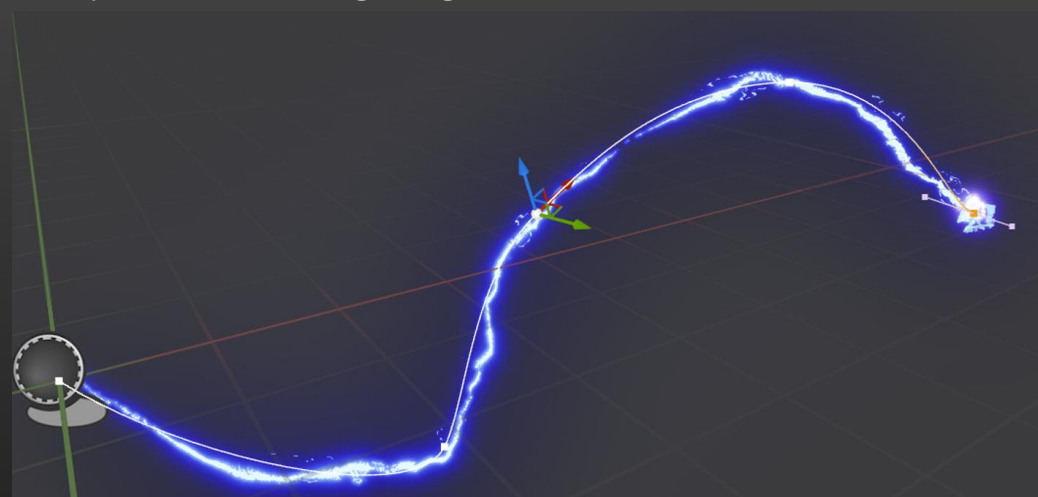
工夫点③ レールガンの射撃演出

プラズマ表現

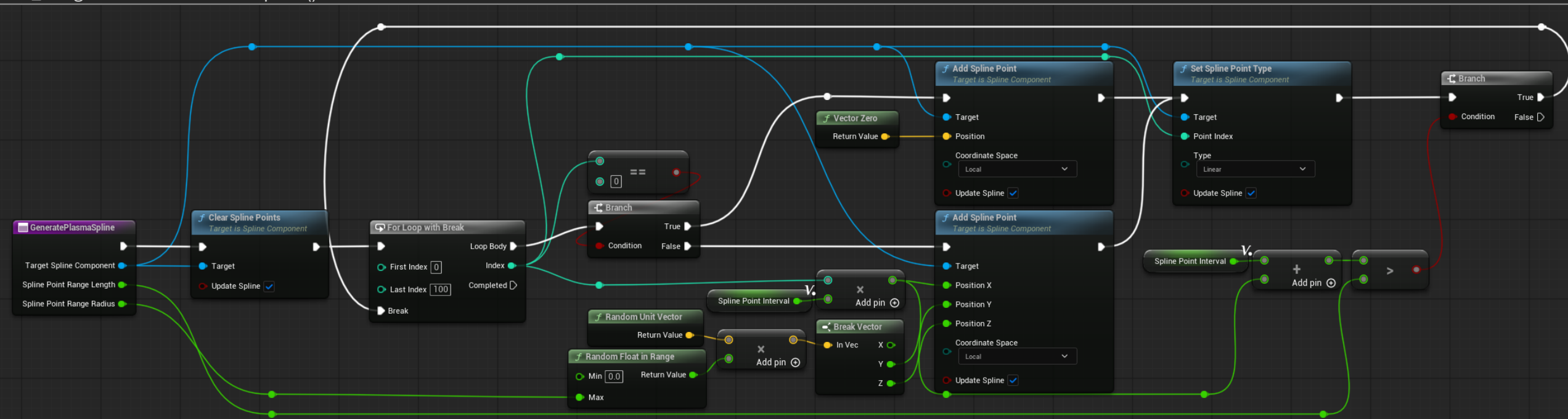
無料アセットでスプラインに沿ってプラズマが走るものがあったので、**そのスプラインのかく点を繰り返しランダム生成させる処理を作る**ことで、プラズマを再現

ランダムに生成した単位ベクトルRandomUnitVectorにランダムな半径RandomFloatInRangeをかけることで、指定範囲内にランダムな点を生成できる。

BP_SplineVFX_ElectricLightning



BP_Railgun.GeneratePlasmaSpline()



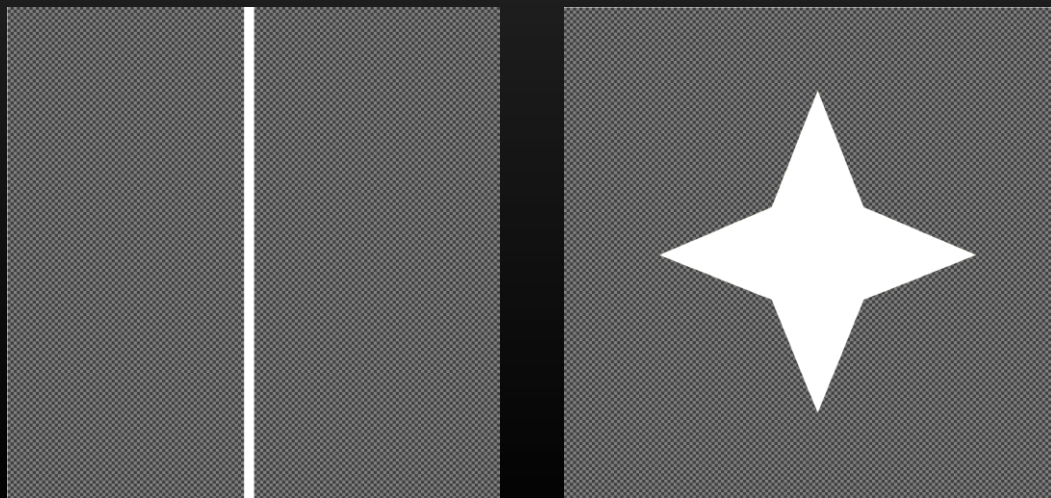
工夫点③ レールガンの射撃演出

着弾エフェクト

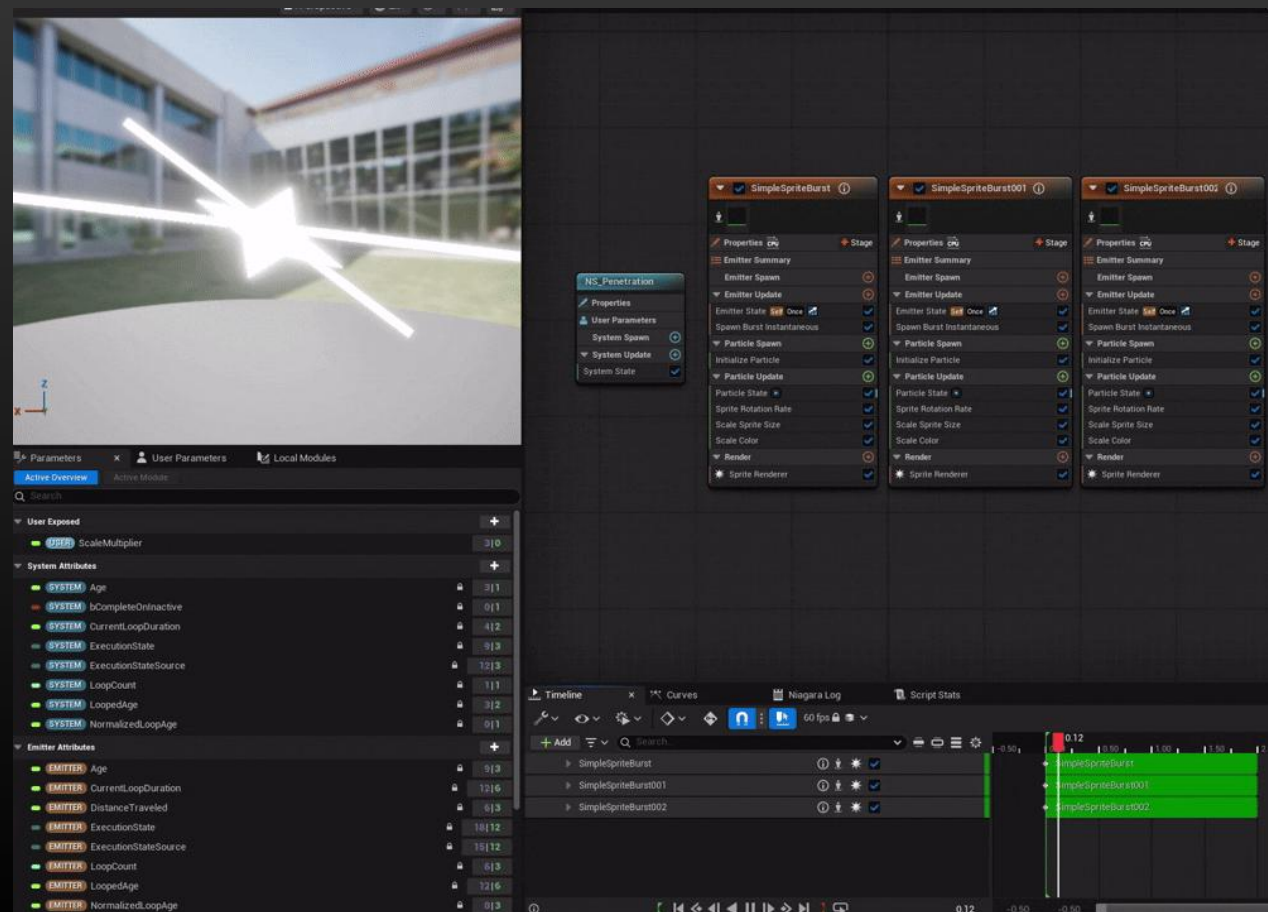
もともとなる画像は生成AIに生成してもらったものを微調整し、それらをNiagaraで動きを調整して実装

- ・ 横方向に長くすることで存在感を演出
- ・ 回転させることで、レールガンで貫いた感覚を表現

使用したイラスト



NS_Penetration



工夫点③ レールガンの射撃演出

木箱の破壊演出

Geometry Collectionを使用することで、破壊されたときに破片が飛び散るように調整し、レールガンの威力を際立たせた。

ベクトル計算の組み込み

木箱・木箱の破片が弾道を中心に円形外向きに広がるようにベクトルでImpulseで飛ばす方向を計算

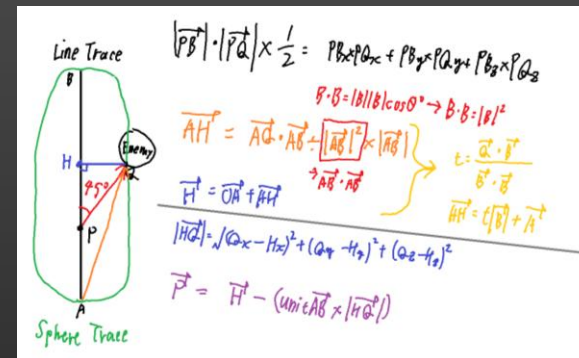
Geometry Collectionの挙動調整に難航

Geometry Collectionが少し接触するだけで崩れたりどこかへ吹き飛んでしまうため、StaticMeshとの競合などの調整に非常に苦労した。とりあえずは物理計算する対象を破壊された瞬間にStaticMeshからGeometryCollectionに入れ替えることで解決した。

BP_Railgun.OnDeath() 物理計算の切り替え部分



計算メモ



円形外向きに押し出される様子



まとめ・今後の展望

体験したこと

- インターフェース, 継承を用いて拡張性の高い設計を実装できた
- さらにその設計がツールによる追加処理にて効果を発揮した！
- ツールを作り、作業を効率化する大切さを実感した！

今後の展望

- オブジェクト指向以外の設計手法を学ぶ
- ツールをさらに細かく調整できるよう修正する
- GCに関してはまだ挙動がおかしいため、調整していきたい